DECA 3.3 - Pipelining and Interrupts

18 March 2023 17:09

Before starting the work on lab 3 of DECA section 3, we are told to make sure we have the most up-to-date versions of both Issie, and the EEP1 assembler. I have updated both of them (git pull for their respective GitHub repositories).

After opening the lab3p.dprj Issie project, we can note that, as explained in the lab 3 handout, the top-level sheet for this EEP1 CPU design now incorporates a State Machine.
State machines are an essential part of CPU design. Before starting this lab, I did some research about the use of state machines in CPU design. I found it very interesting and useful to read through the In my research, I came across the **Dining Philosopher's Problem**, as expressed in the Modern Operating Systems textbook by A. Tanenbaum and Herbert Bos. The problem was first posed by Edsger Dijkstra in 1965, and goes as follows: five philosophers are seated around a round table, each with a bowl of food in front of them. In order to eat this food, they must use two forks. Let us imagine that it is not possible to eat with only one fork - two are necessary to eat. However, there are only a total of 5 forks on the table, one between each plate.

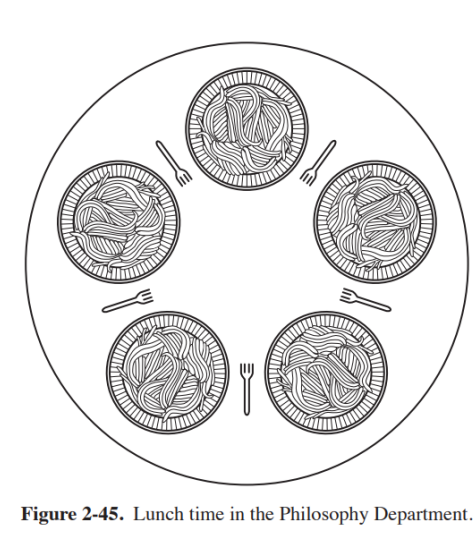


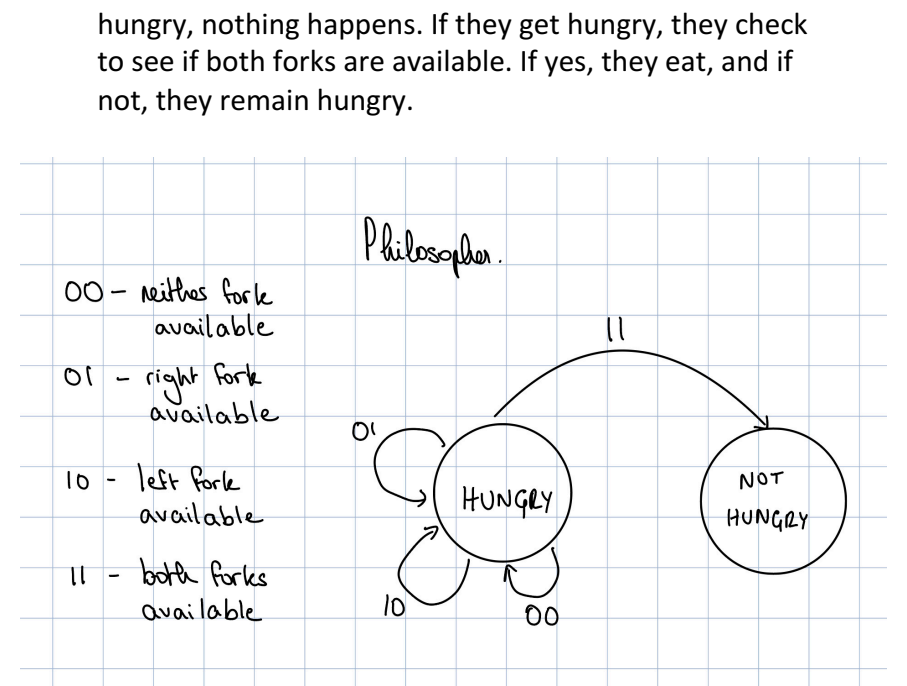
Figure 2-45. Lunch time in the Philosophy Department.

This illustration, taken from the aforementioned textbook, is a useful tool in visualising the scenario. The problem is obvious: all of the philosophers cannot eat at the same time, since that would require 10 forks. (Just like how a CPU's resources are limited - it cannot do everything at once).
At any given time, a philosopher is either eating or not eating. If a philosopher gets hungry, they attempt to eat by picking up the two forks either side of them. If they succeed, or in other words if both forks are available, they will eat for a certain amount of time, then proceed to put the forks back. However, if one or both of the forks is not available (in other words, if another philosopher is currently using them to eat), then the philosopher tries again later.

The "try again later" method works in small-scale applications. For instance, in ethernet local area networks, if two computers attempt to send a packet at the same time, then they both wait a random amount of time, and try again. However, the problem with this solution is that one would prefer a solution which always works, reliably. This might be important in time-critical applications, or in situations where other resources are limited.

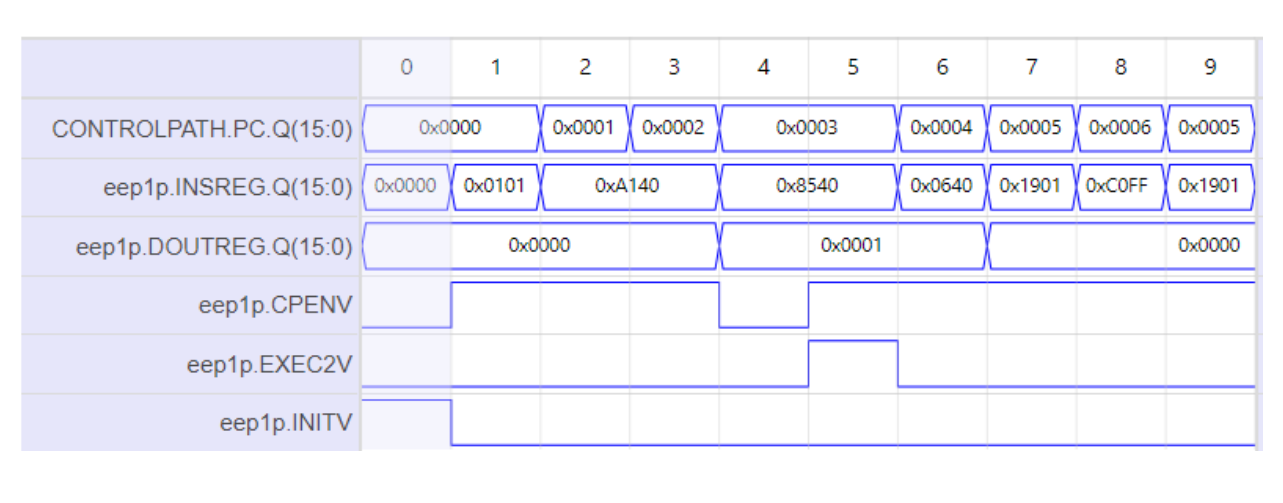
At this point, the notion of mutexes and semaphores might be introduced, but I do not believe I understand these concepts well enough to write about them in my logbook. Here is my limited understanding:
A mutex is an object that serves as a lock, used for mutual exclusion among threads. A semaphore is an object used to limit access to a common resource by multiple threads.

The link with state machines is that this situation can be represented as one. A philosopher exists. If they are not hungry, nothing happens. If they get hungry, they check to see if both forks are available. If yes, they eat, and if not, they remain hungry.

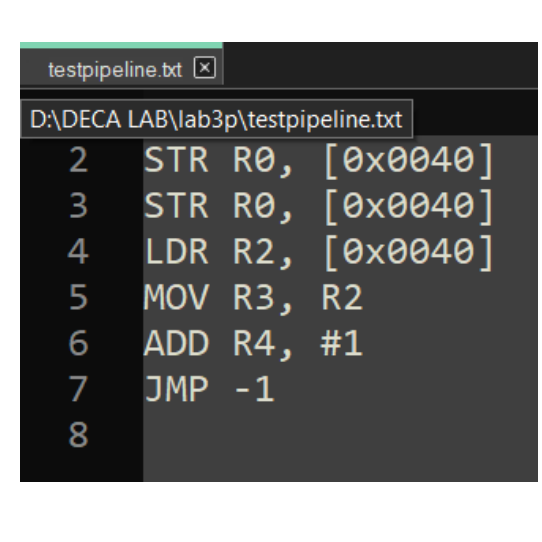

Philosopher.
00 – neither fork available
01 – right fork available
10 – left fork available
11 – both forks available
HUNGRY
NOT HUNGRY
01
10
00
11

Task 0 - Checks: Before starting the practical work ensure that you have updated eepassembler to v2.1. Show the first 9 clock cycles of the ee1p sheet of the lab3p design running with pipetask.ram program in the waveform simulator. Set the simulator to view INSREG.Q, DOUTREG.Q, CPENV, EXEC2V, INITV, as well as the PC and registers. View the pipetask.txt file to see the instructions simulated.

These are the waveforms we see:


0 1 2 3 4 5 6 7 8 9
CONTROLPATH.PC.Q(15:0)
0x0000 0x0001 0x0002 0x0003 0x0004 0x0005 0x0006 0x0005
eep1p.INSREG.Q(15:0)
0x0000 0x0101 0xA140 0x8540 0x0640 0x1901 0xC0FF 0x1901
eep1p.DOUTREG.Q(15:0)
0x0000 0x0001 0x0000
eep1p.CPENV
eep1p.EXEC2V
eep1p.INITV

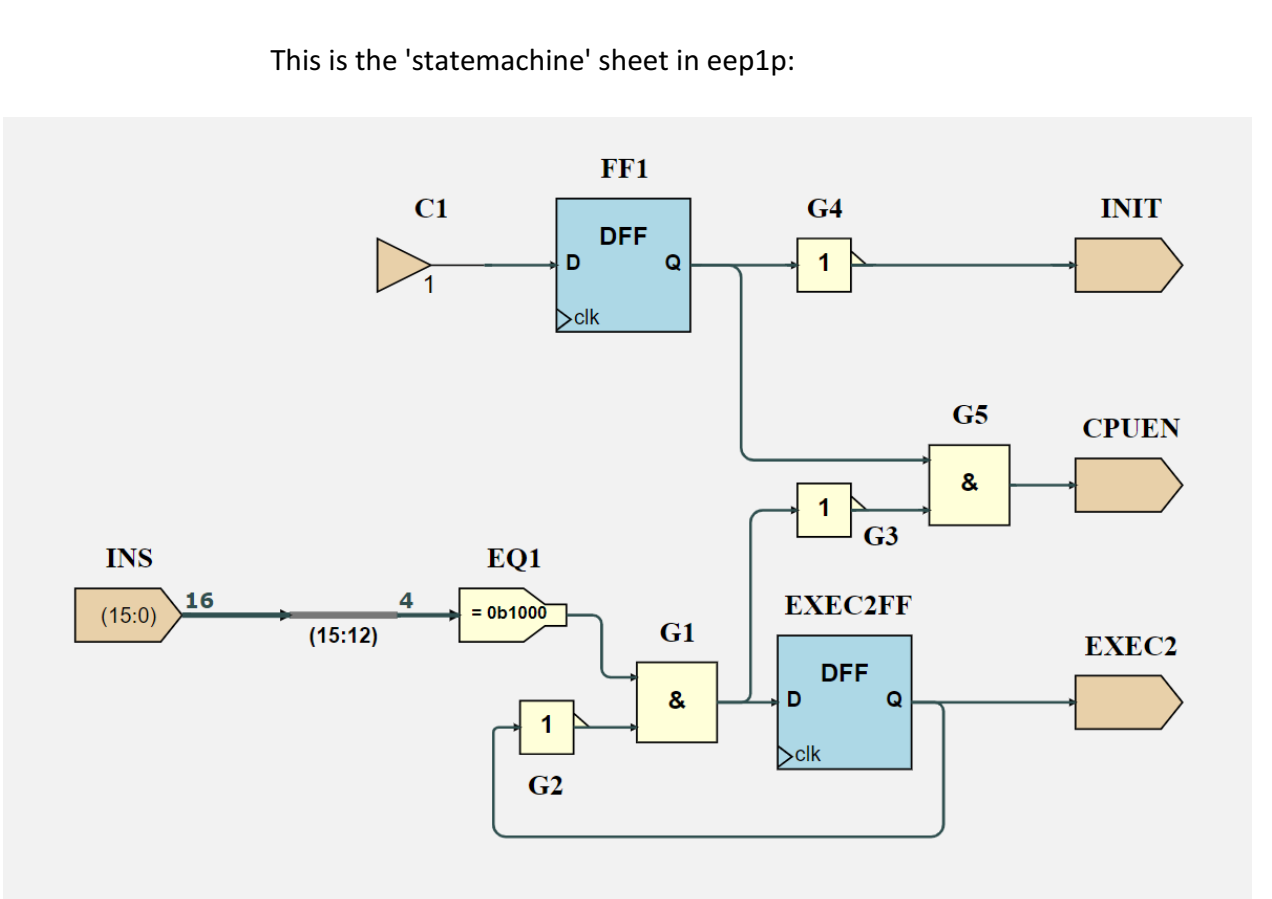
These are the instructions which we were running: (testpipeline.txt)



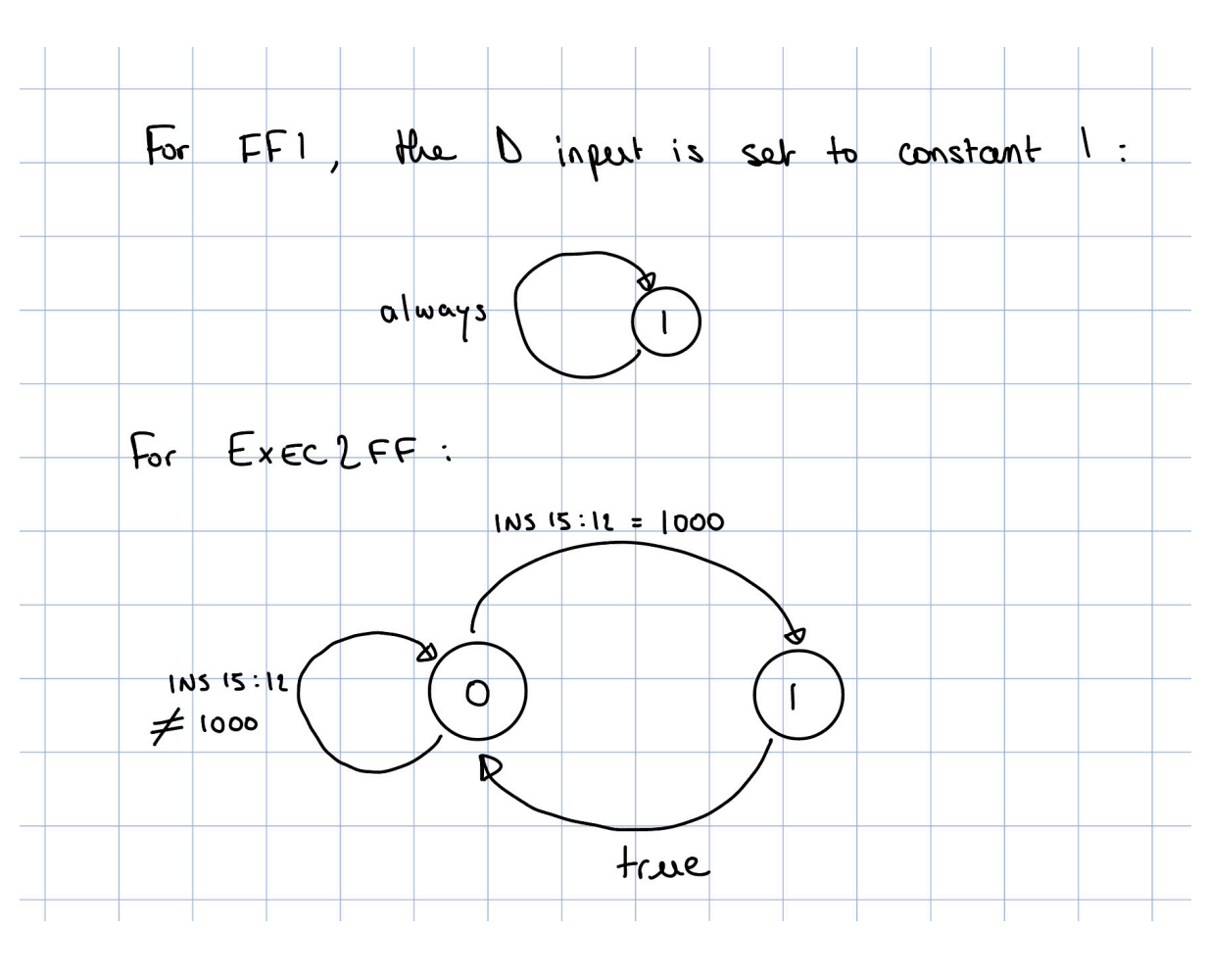
```
D:\DECA LAB\lab3p\testpipeline.txt
STR R0, [0x0040]
STR R0, [0x0040]
LDR R2, [0x0040]
MOV R3, R2
ADD R4, #1
JMP -1
```

Task 1 - Load the 'statemachine' sheet of ee1p to see the design of SM1. The two flip-flops implement two independent state machines. Write two separate state diagrams, one for FF1, and the other for EXEC2FF. Explain the INITV waveform from your FF1 state diagram, say how it affects clock cycle 0 in Figure 3. EXEC2V is high during EXEC2 - determine from your EXEC2FF state diagram when this happens and check that this agrees with Figure 3.

This is the 'statemachine' sheet in eep1p:


C1
FF1
DFF
D
Q
clk
G4
1
INIT
G5
&
CPUEN
1
G3
INS
(15:0)
16
4
(15:12)
EQ1
= 0b1000
G1
&
EXEC2FF
DFF
EXEC2
1
G2

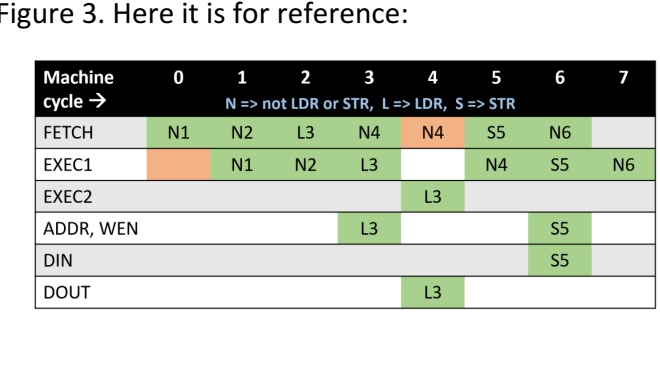
The two flip-flops implement two independent state machines, one for FF1, and the other for EXEC2FF. The first task is to write to separate state diagrams, one for each state machine.


For FF1, the D input is set to constant 1 :
always
1
For Exec2FF :
INS 15:12 = 1000
INS 15:12 ≠ 1000
0
1
true

INITV is true for the first clock cycle for which the CPU is active, and then false after that.

We are told that EXEC2V is high during EXEC2. We are then told to determine when that is, from our previous state diagrams. Since we know that EXEC2V is high during EXEC2, we can determine that EXEC2V is high when INS(15:12) = 1000.

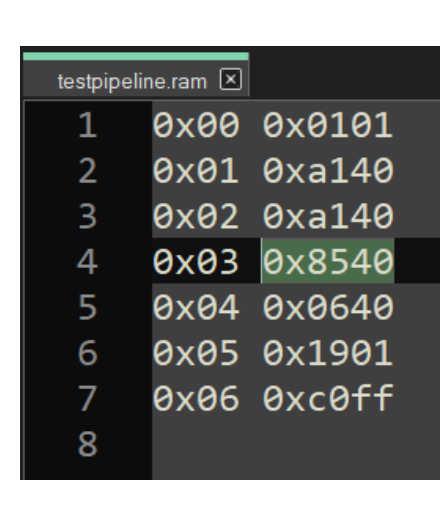
We are then asked to check whether this agrees with Figure 3. Here it is for reference:



Machine cycle →	0	1	2	3	4	5	6	7
	N => not LDR or STR, L => LDR, S => STR							
FETCH	N1	N2	L3	N4	N4	S5	N6	
EXEC1		N1	N2	L3		N4	S5	N6
EXEC2					L3			
ADDR, WEN				L3			S5	
DIN							S5	
DOUT					L3			

Figure 3: EEP1P pipeline diagram

From this figure, we observe that EXEC2 is true at instruction 4. If we check the assembled .ram file, we can see what this instruction is.

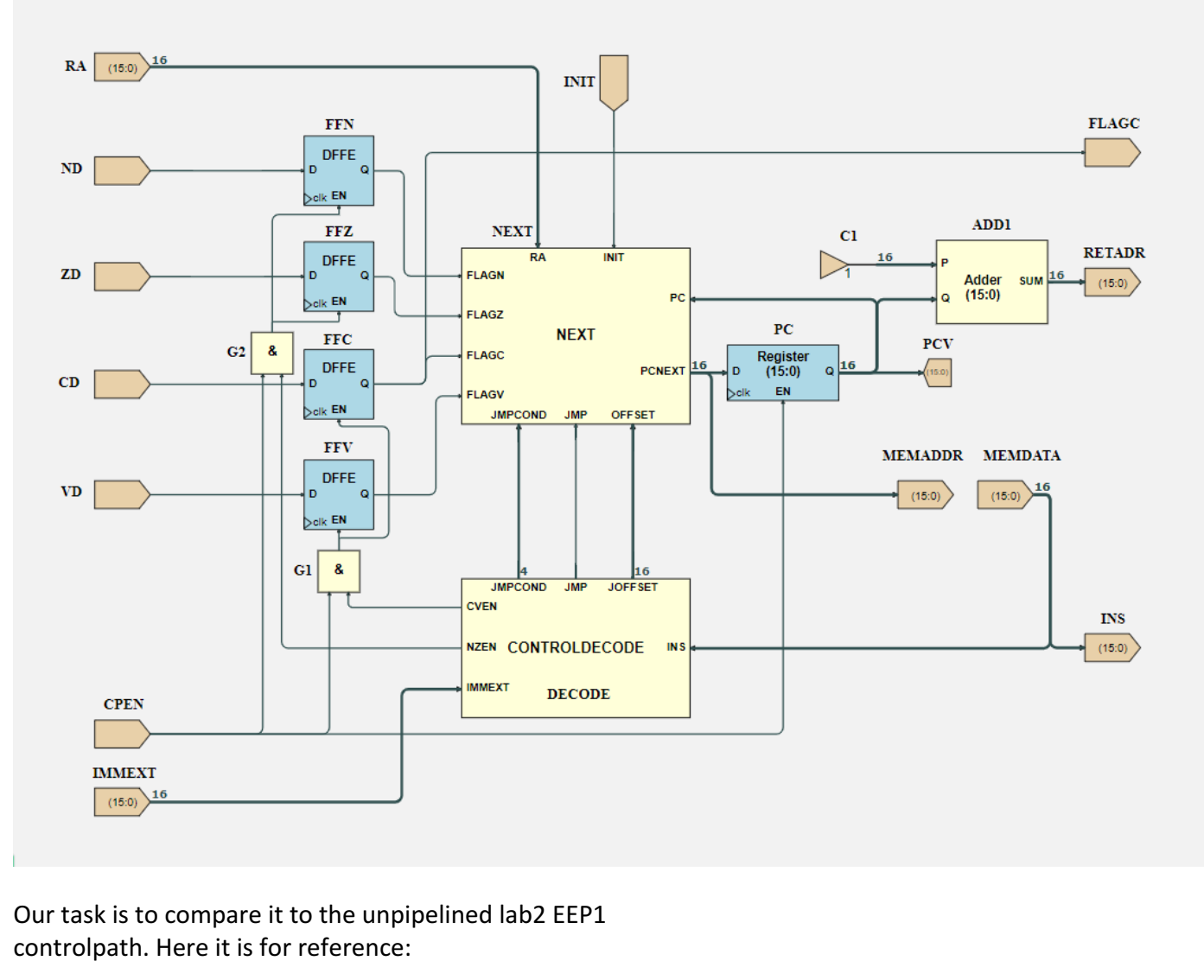


```
testpipeline.ram
0x00 0x0101
0x01 0xa140
0x02 0xa140
0x03 0x8540
0x04 0x0640
0x05 0x1901
0x06 0xc0ff
```

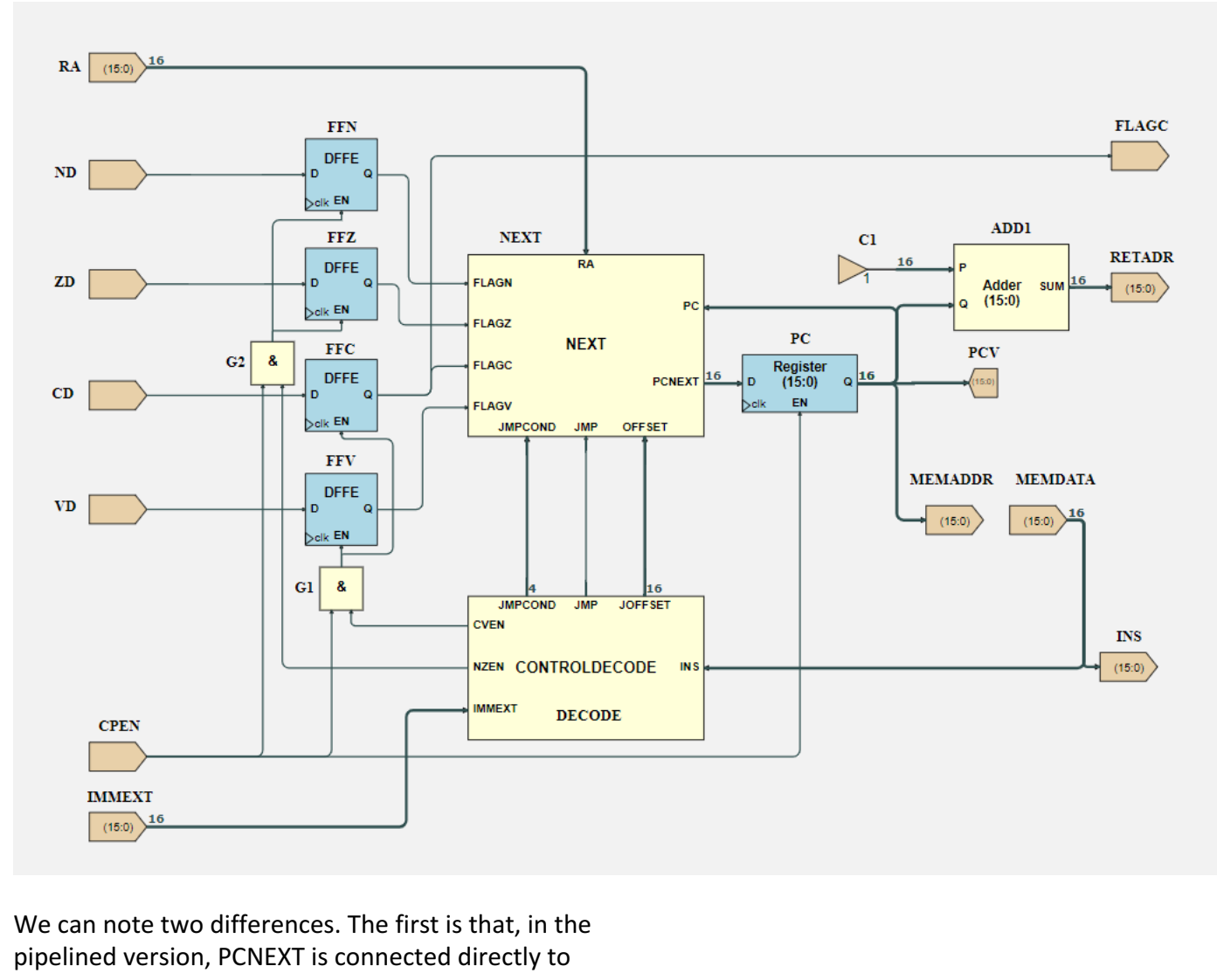
In hexadecimal, the instruction code is 0x8540. If we convert this into binary, we get 0b1000010101000000. Notice that bits 12 through 15 are 1000. We can therefore conclude that our previous state diagram was correct - when the 4 most significant bits of the instruction word were 1000, EXEC2 was true (and therefore so was EXEC2V).

Task 2 - Look at MEMADDR on the controlpath sheet of lab3p and compare it with the unpipelined lab2 EEP1 controlpath. Explain how the change to MEMADDR allows the correct next instruction to be fetched. You will find it useful to compare data delays through the two circuits between PCNEXT and INS.

Here is the controlpath sheet from the lab3p model.

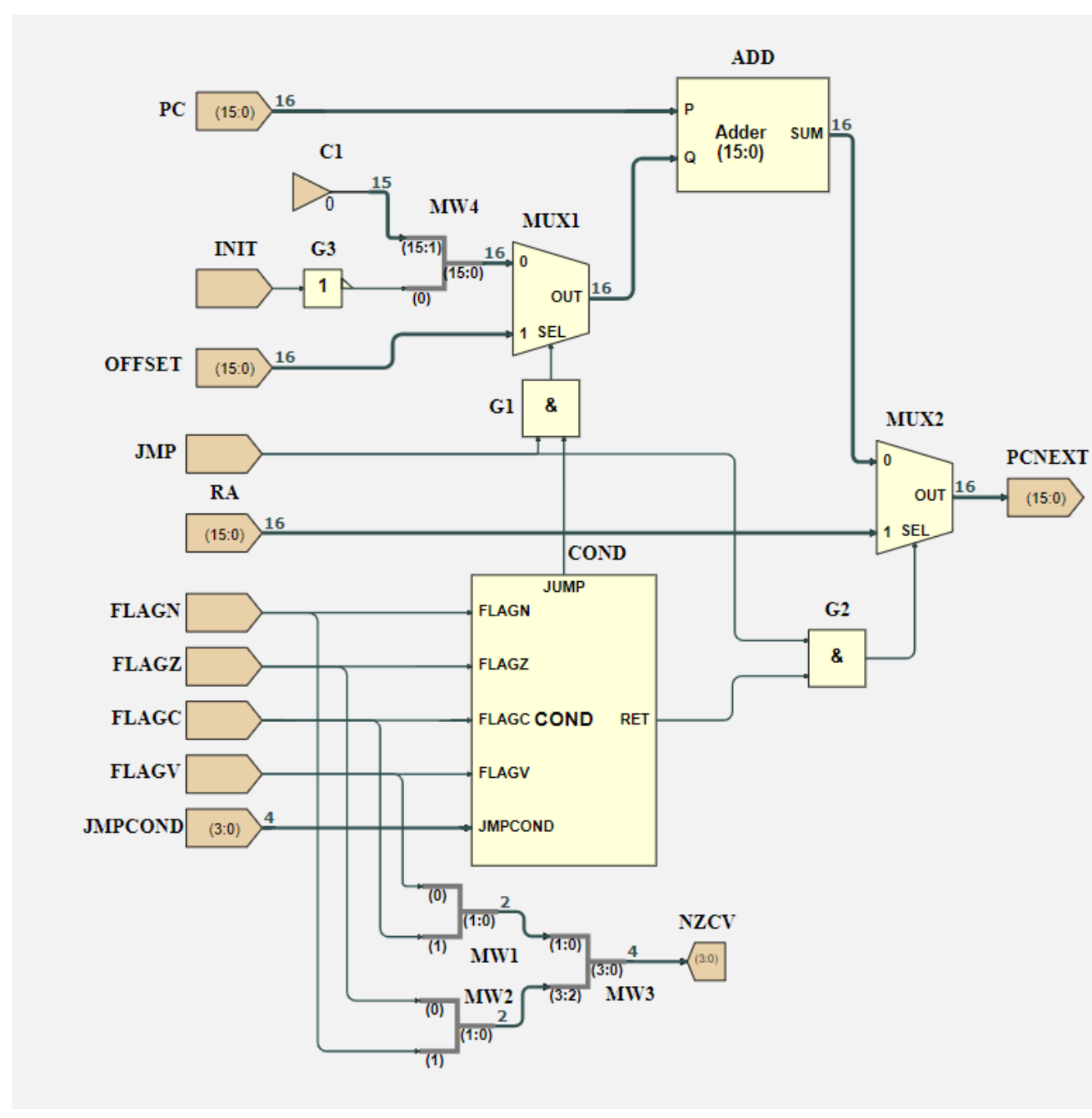

RA
ND
ZD
CD
VD
FFN
FFZ
FFC
FFV
DFFE
INIT
NEXT
CONTROLDECODE
DECODE
CPEN
IMMEXT
FLAGC
ADD1
Adder (15:0)
RETADR
PC
PCV
MEMADDR
MEMDATA
INS

Our task is to compare it to the unpipelined lab2 EEP1 controlpath. Here it is for reference:


RA
ND
ZD
CD
VD
FFN
FFZ
FFC
FFV
DFFE
NEXT
CONTROLDECODE
DECODE
CPEN
IMMEXT
FLAGC
ADD1
Adder (15:0)
RETADR
PC
PCV
MEMADDR
MEMDATA
INS

We can note two differences. The first is that, in the pipelined version, PCNEXT is connected directly to MEMADDR, whereas in the unpipelined version, PCNEXT goes through the PC register before being output to MEMADDR. The second difference is that the pipelined controlpath also has an INIT input for its NEXT block.

In order to figure out what the purpose of the INIT input is, we can look at the NEXT block:


PC
(15:0)
C1
INIT
G3
MW4
MUX1
OUT
SEL
OFFSET
G1
&
ADD
Adder (15:0)
SUM
MUX2
PCNEXT
(15:0)
JMP
RA
FLAGN
FLAGZ
FLAGC
FLAGV
JMPCOND
(3:0)
JUMP
COND
RET
G2
&
MW1
MW2
MW3
NZCV

Given this sheet, we can make a truth table for MUX1:

MUX1	JMP	COND
! INIT (16 bit)	0	0
! INIT (16 bit)	0	1
! INIT (16 bit)	1	0
OFFSET	1	1

EEP1 Interrupt mechanism

Task 3 - Simulate the eep1 sheet of lab3i running with instruction memory loaded from iotest1.ram in the step simulator. Run it for a few cycles to check its normal loop. Then manually toggle IRQ0TOGGLE and observe how this creates an IRQ0 request, which is detected by the program and reset, adding 1 to R2. This iotest1.txt program, using memory-mapped i/o and a software loop, can therefore count the number of edges on IRQ0TOGGLE.

Code must be executed in a deterministic order - there is no way of interfacing easily with external hardware if things happen in asynchronous matter. We don't want to have conditional jumps everywhere.

Polling: dedicated program which waits for hardware to do something and interfaces with it when it does. This implementation requires no extra hardware, which is an advantage, but comes at the cost of being inefficient.

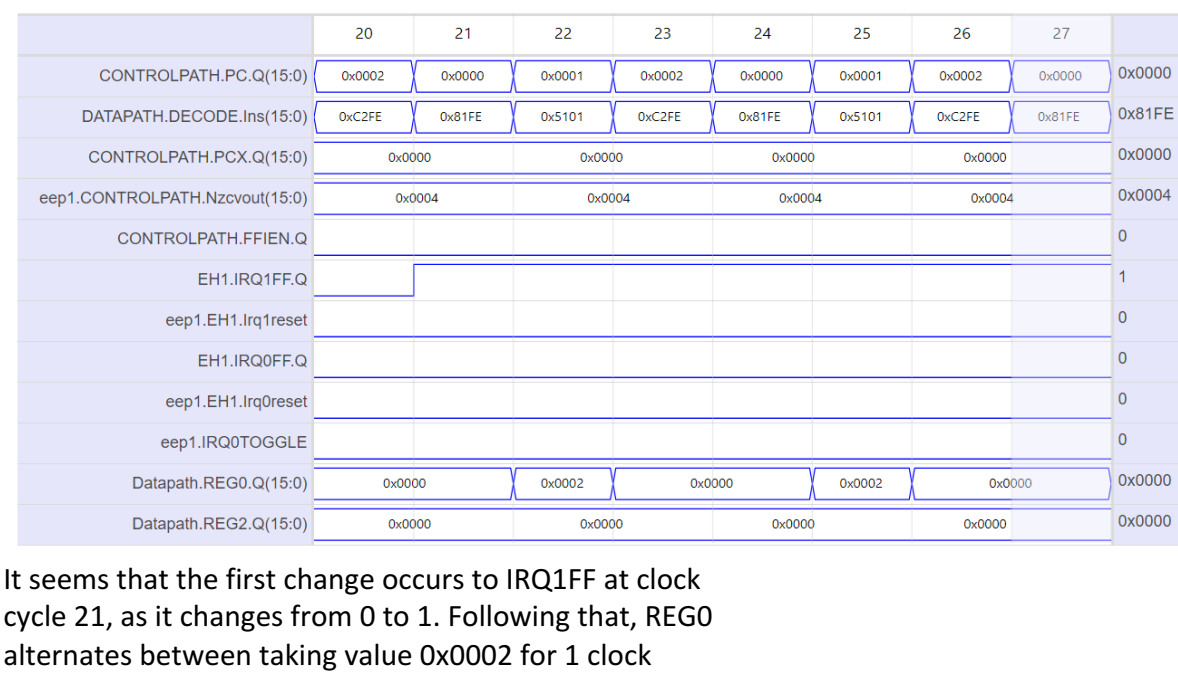
Interrupts: extra hardware so that can jump when needed to a different part of the program; execute it and then come back as if nothing had happened.

A single hardware device requires the CPU to execute the ISR (interrupt service routine) repeatedly. The time at which each execution happens is determined by the hardware. The CPU must respond to an interrupt request from the device ASAP.

We need a special register that the base program is not allowed to use that can hold the program counter until we need it.
In eep1, we do this by creating a new register called PCX that stores the PC at the beginning of the interrupt, and writes it back to PC at the end of the interrupt.

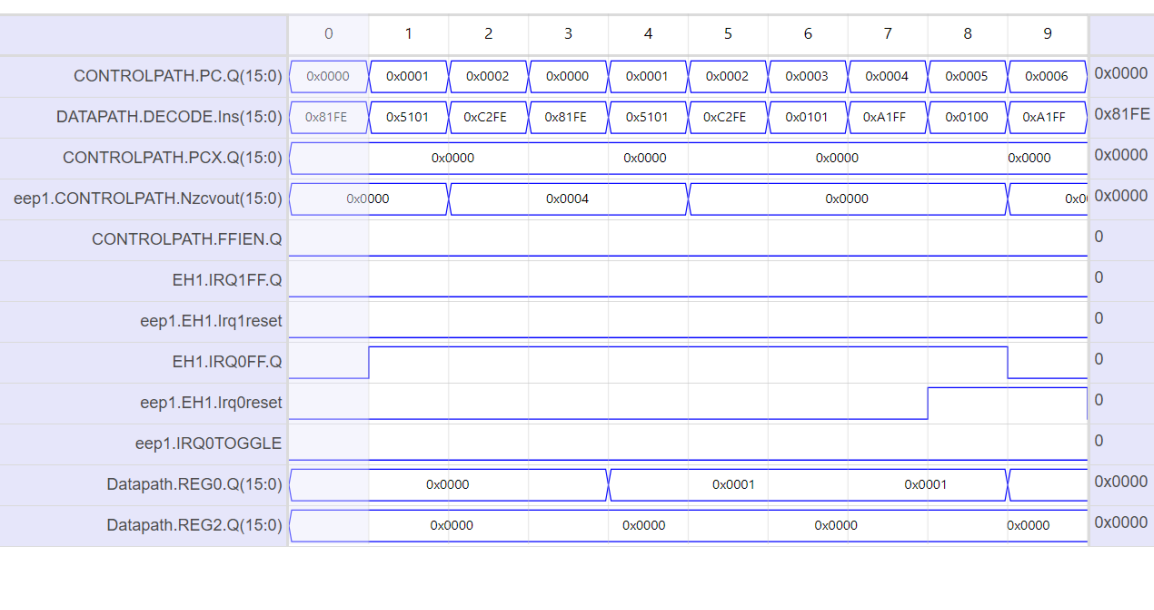
We also need a flipflop that controls interrupts (if 1: can happen, if 0 cannot happen).

When simulating the lab3i CPU running the iotest1.ram file, we observe the following behaviour:


CONTROLPATH.PC.Q(15:0)
DATAPATH.DECODE.Ins(15:0)
CONTROLPATH.PCX.Q(15:0)
eep1.CONTROLPATH.Nzcvout(15:0)
CONTROLPATH.FFIEN.Q
EH1.IRQ1FF.Q
eep1.EH1.Irq1reset
EH1.IRQ0FF.Q
eep1.EH1.Irq0reset
eep1.IRQ0TOGGLE
Datapath.REG0.Q(15:0)
Datapath.REG2.Q(15:0)

It seems that the first change occurs to IRQ1FF at clock cycle 21, as it changes from 0 to 1. Following that, REG0 alternates between taking value 0x0002 for 1 clock cycle, then 0x0000 for 2 clock cycles, then repeating.

If I set IRQ0TOGGLE to constant 1, then IRQ0FF becomes 1 at clock cycle 1 and stays at 1 for 8 clock cycles, going back to 0 at clock 9.


CONTROLPATH.PC.Q(15:0)
DATAPATH.DECODE.Ins(15:0)
CONTROLPATH.PCX.Q(15:0)
eep1.CONTROLPATH.Nzcvout(15:0)
CONTROLPATH.FFIEN.Q
EH1.IRQ1FF.Q
eep1.EH1.Irq1reset
EH1.IRQ0FF.Q
eep1.EH1.Irq0reset
eep1.IRQ0TOGGLE
Datapath.REG0.Q(15:0)
Datapath.REG2.Q(15:0)

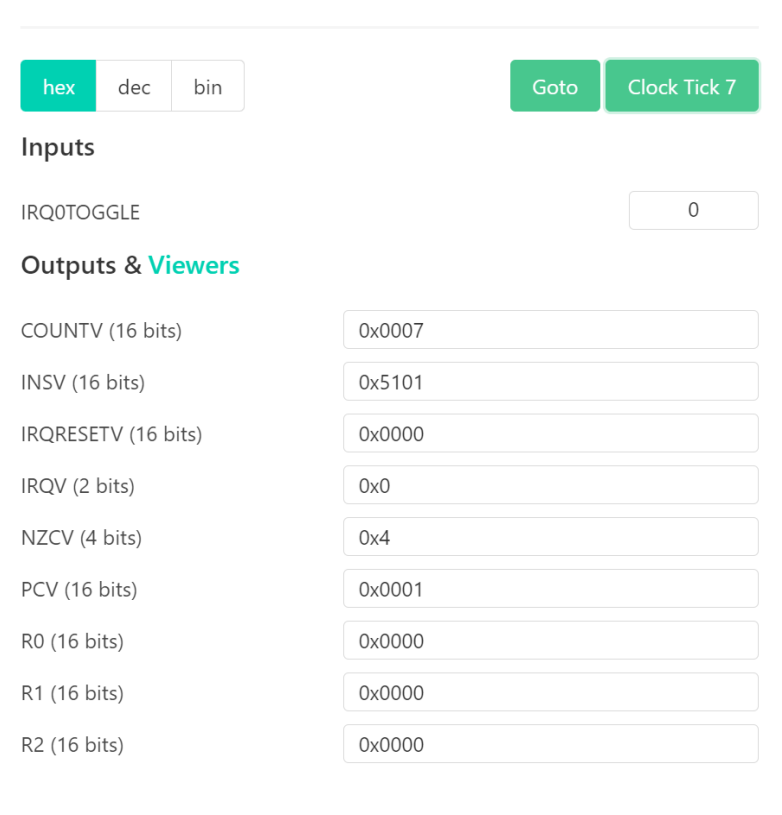
By running a step simulation, we can better visualise what is going on.

Clock cycle	PCV	REG0	IRQ0TOGGLE
0	1	0	0
1	2	0	0
2	0	0	0
3	1	0	0
4	2	0	0
5	0	0	0
7	1	0	0
8	2	0	0
9	0	0	0
…	…	…	…
20	0	0	0
21	1	2	0
22	2	0	0
23	0	0	0
24	1	2	0
25	2	0	0
26	0	0	0
27	1	2	0

From the beginning onwards, PCV cycles through values 0, 1, and 2. REG0 is set to 0 until clock cycle 21, at which point it starts alternating between 0x0000 for 2 clock cycles, and 0x0002 for 1 clock cycle, which is the same behaviour as we had before.

If now we run the same program and we manually toggle IRQ0TOGGLE, we observe:

Before toggling:


hex
dec
bin
Goto
Clock Tick 7

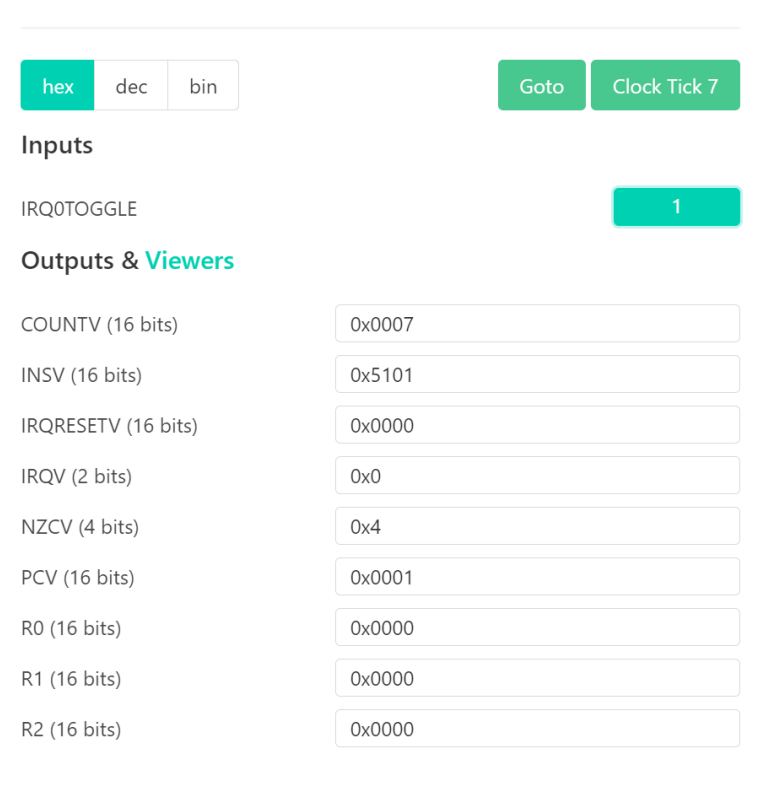
Inputs

IRQ0TOGGLE 0

Outputs & Viewers

COUNTV (16 bits)	0x0007
INSV (16 bits)	0x5101
IRQRESETV (16 bits)	0x0000
IRQV (2 bits)	0x0
NZCV (4 bits)	0x4
PCV (16 bits)	0x0001
R0 (16 bits)	0x0000
R1 (16 bits)	0x0000
R2 (16 bits)	0x0000

When toggled, change occurs at next clock cycle:


hex
dec
bin
Goto
Clock Tick 7

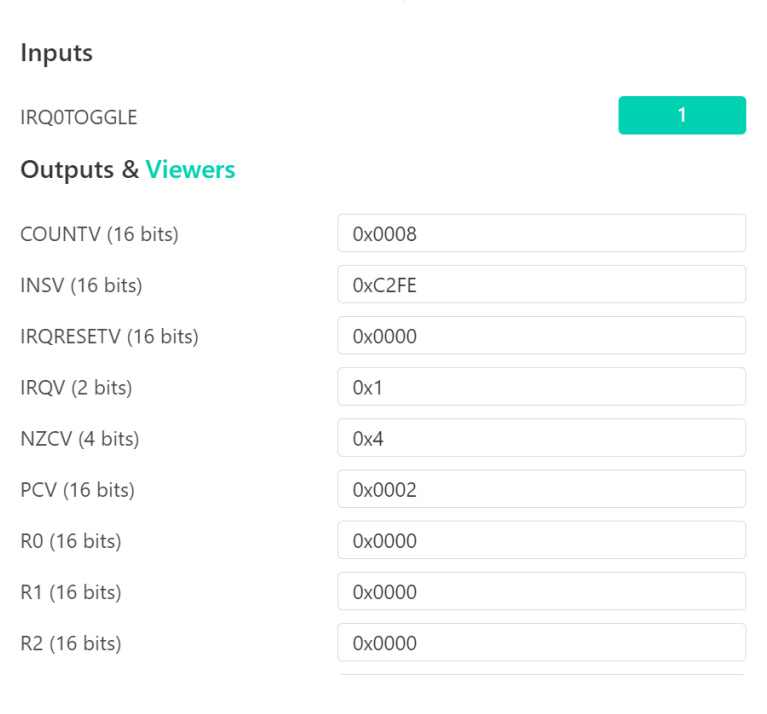
Inputs

IRQ0TOGGLE 1

Outputs & Viewers

COUNTV (16 bits)	0x0007
INSV (16 bits)	0x5101
IRQRESETV (16 bits)	0x0000
IRQV (2 bits)	0x0
NZCV (4 bits)	0x4
PCV (16 bits)	0x0001
R0 (16 bits)	0x0000
R1 (16 bits)	0x0000
R2 (16 bits)	0x0000

IRQV has been set to 1: an IRQ0 request has been made.



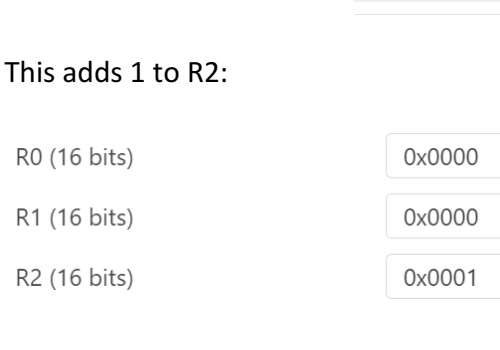
Inputs

IRQ0TOGGLE 1

Outputs & Viewers

COUNTV (16 bits)	0x0008
INSV (16 bits)	0xC2FE
IRQRESETV (16 bits)	0x0000
IRQV (2 bits)	0x1
NZCV (4 bits)	0x4
PCV (16 bits)	0x0002
R0 (16 bits)	0x0000
R1 (16 bits)	0x0000
R2 (16 bits)	0x0000

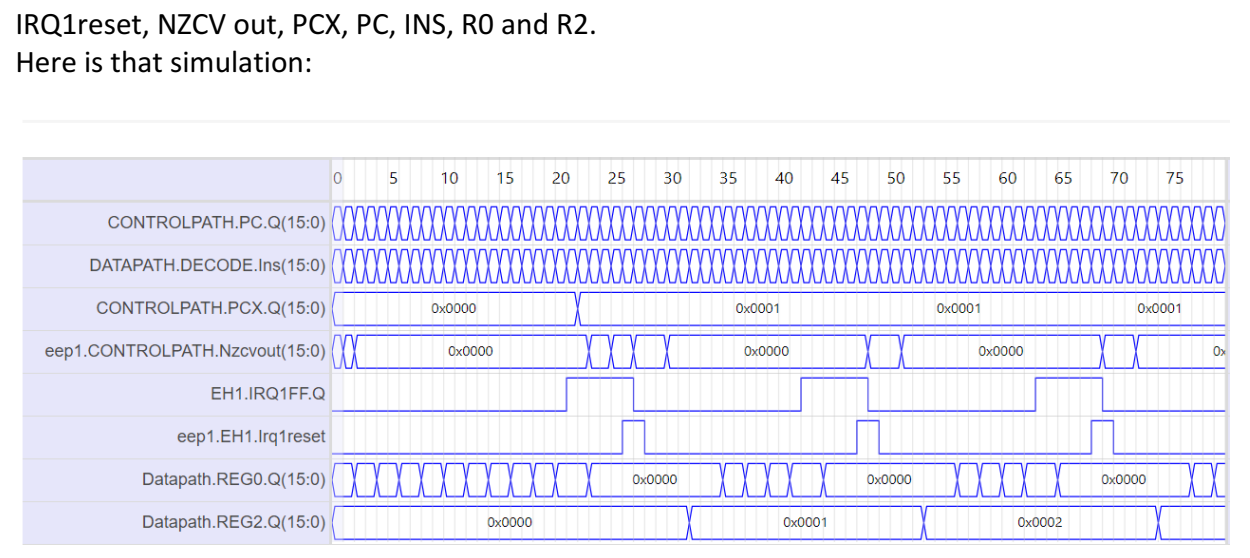
This adds 1 to R2:



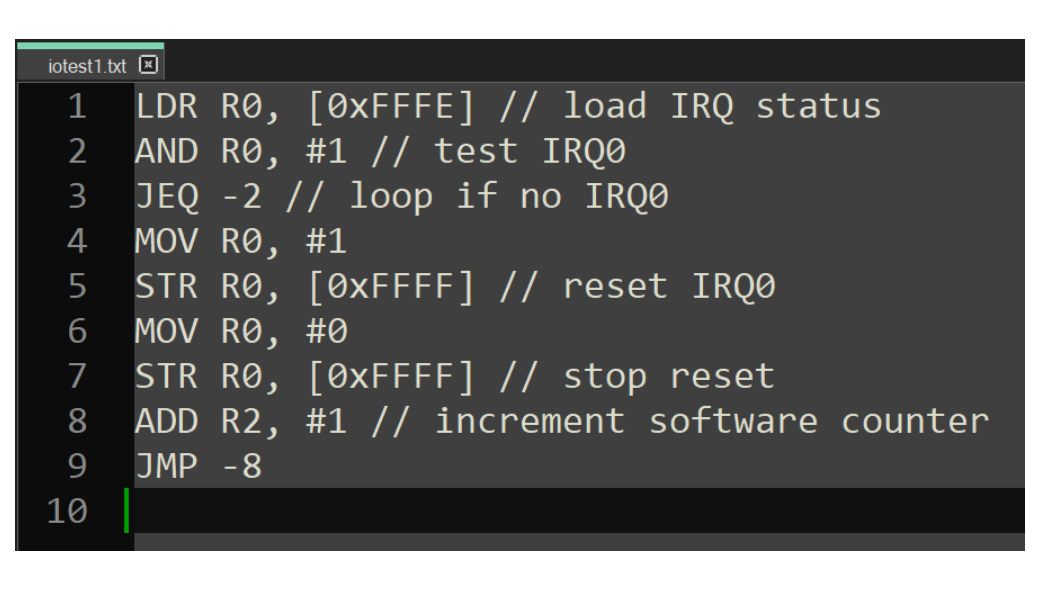
R0 (16 bits)	0x0000
R1 (16 bits)	0x0000
R2 (16 bits)	0x0001

Task 4 - Simulate eep1 sheet of lab3i running with int-test1.ram in the waveform simulator. View IRQ1, IRQ1reset, NZCVout, PCX in addition to PC, INS, R0, R2. Trace the EEP1I execution from when IRQ1 goes high through the interrupt jump to ISR, until the RETINT instruction after which the base program continues execution. Note the waveforms. For every change you see work out which instruction it comes from, or if it comes from the interrupt jump mechanism. Looking at the code and your waveforms explain why the base program PC, flags, and R0 are unaffected by the execution of the Interrupt Service Routine.

The first task is to simulate the eep1 sheet of lab3i running with int-test1.ram. We are interested in IRQ1, IRQ1reset, NZCV out, PCX, PC, INS, R0 and R2.
Here is that simulation:


0 5 10 15 20 25 30 35 40 45 50 55 60 65 70 75
CONTROLPATH.PC.Q(15:0)
DATAPATH.DECODE.Ins(15:0)
CONTROLPATH.PCX.Q(15:0)
eep1.CONTROLPATH.Nzcvout(15:0)
EH1.IRQ1FF.Q
eep1.EH1.Irq1reset
Datapath.REG0.Q(15:0)
Datapath.REG2.Q(15:0)

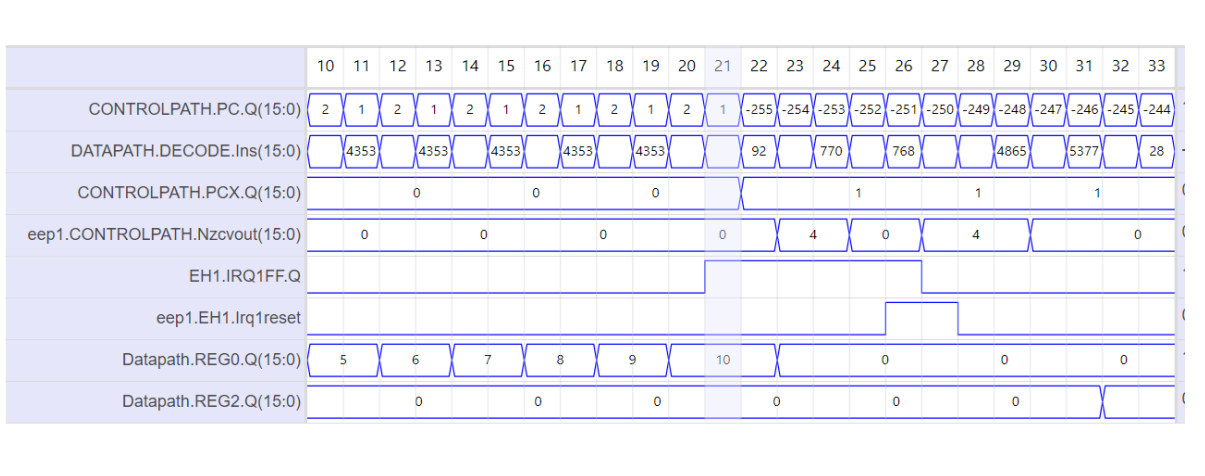
Here is the code we are running:



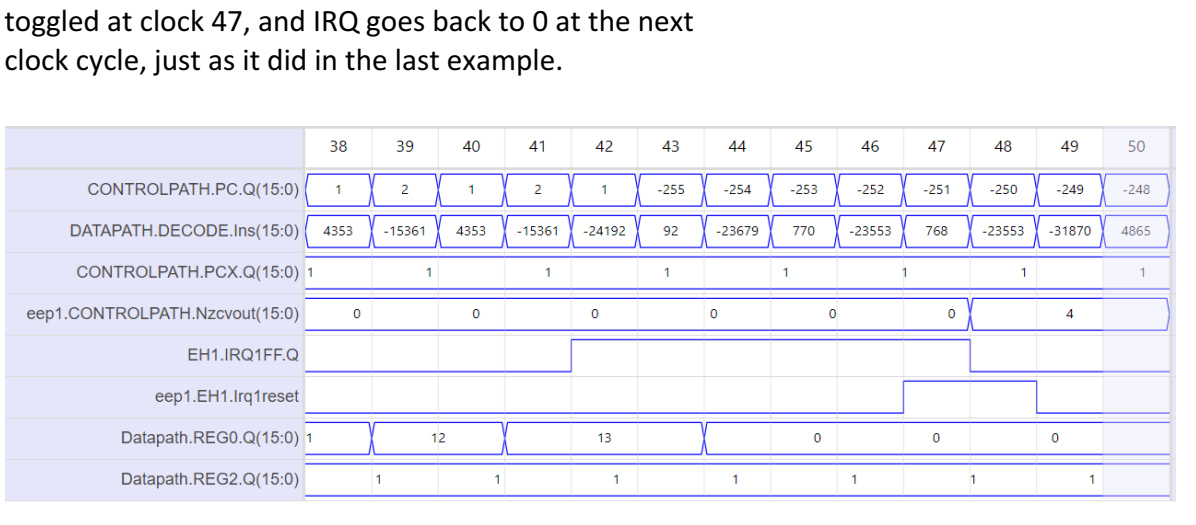
```
iotest1.txt
LDR R0, [0xFFFE] // load IRQ status
AND R0, #1 // test IRQ0
JEQ -2 // loop if no IRQ0
MOV R0, #1
STR R0, [0xFFFF] // reset IRQ0
MOV R0, #0
STR R0, [0xFFFF] // stop reset
ADD R2, #1 // increment software counter
JMP -8
```

The first jump occurs at clock cycle 21, and it happens after the instruction LDR R0, [0xFFFE] // load IRQ status.

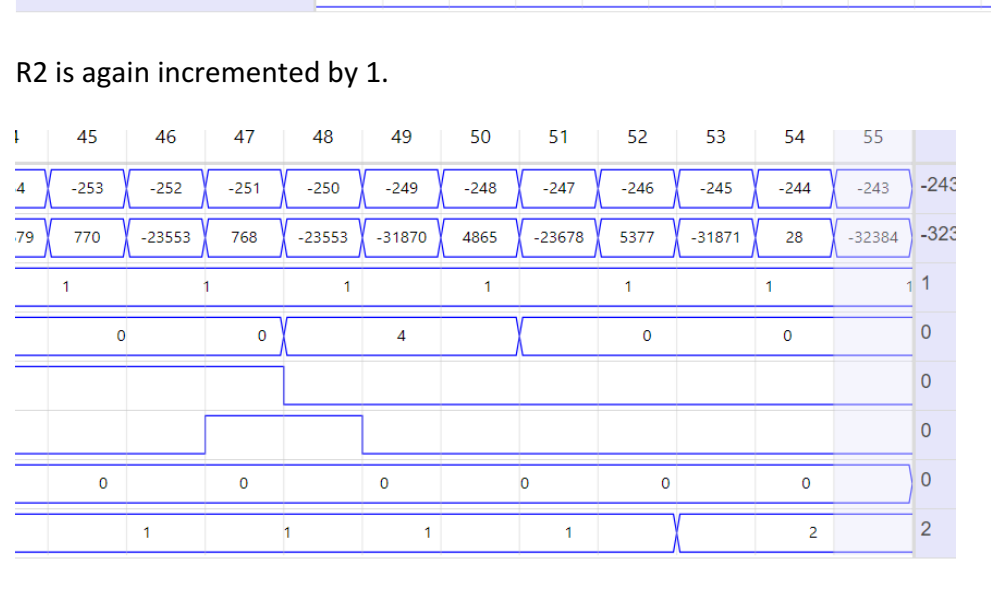
Then, IRQ1reset is triggered at clock cycle 26, and at the next clock cycle, IRQ1 goes back to 0.
R2 is incremented by 1.


10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33
CONTROLPATH.PC.Q(15:0)
DATAPATH.DECODE.Ins(15:0)
CONTROLPATH.PCX.Q(15:0)
eep1.CONTROLPATH.Nzcvout(15:0)
EH1.IRQ1FF.Q
eep1.EH1.Irq1reset
Datapath.REG0.Q(15:0)
Datapath.REG2.Q(15:0)

The next jump occurs at clock cycle 42. IRQ1reset is toggled at clock 47, and IRQ goes back to 0 at the next clock cycle, just as it did in the last example.


38 39 40 41 42 43 44 45 46 47 48 49 50
CONTROLPATH.PC.Q(15:0)
DATAPATH.DECODE.Ins(15:0)
CONTROLPATH.PCX.Q(15:0)
eep1.CONTROLPATH.Nzcvout(15:0)
EH1.IRQ1FF.Q
eep1.EH1.Irq1reset
Datapath.REG0.Q(15:0)
Datapath.REG2.Q(15:0)

R2 is again incremented by 1.


45 46 47 48 49 50 51 52 53 54 55

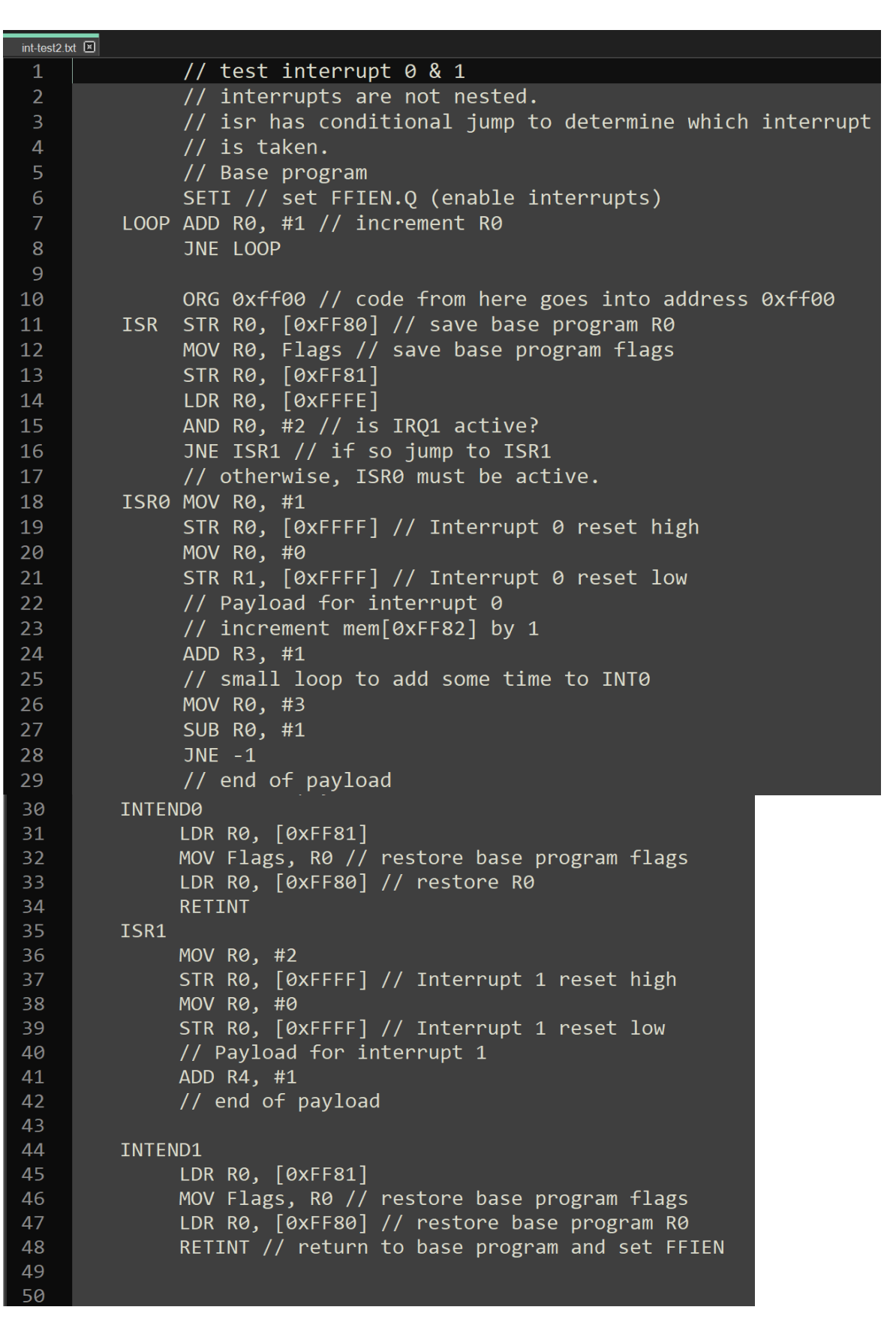
This cycle then repeats predictably every 21 clock cycles.

In order to toggle the reset, we execute MOV R0, #1 then STR R0, [0xFFFF]. This sets the reset toggle to 1. Then, in order to make the reset equal to 0 again, we do: MOV R0, #0 then STR R0, [0xFFFF].

Task 5 - Simulate the eep1x sheet of lab3i running with int-test2.ram in the waveform simulator. View the assembler in int-test2.txt. View FFIEN, IRQ0, IRQ0reset, IRQ1, IRQ1reset, NZCVout, PCX in addition to PC, INS, R0, R2, R3. Note the waveforms. Determine, for the first 50 clock cycles, which are base program, which process INT0, which process INT1. Check that when execution returns to the base program after an interrupt R0 and Flags are unchanged.

We now run the same simulation, but with source code set to int-test2.ram.

This is the code we are running:

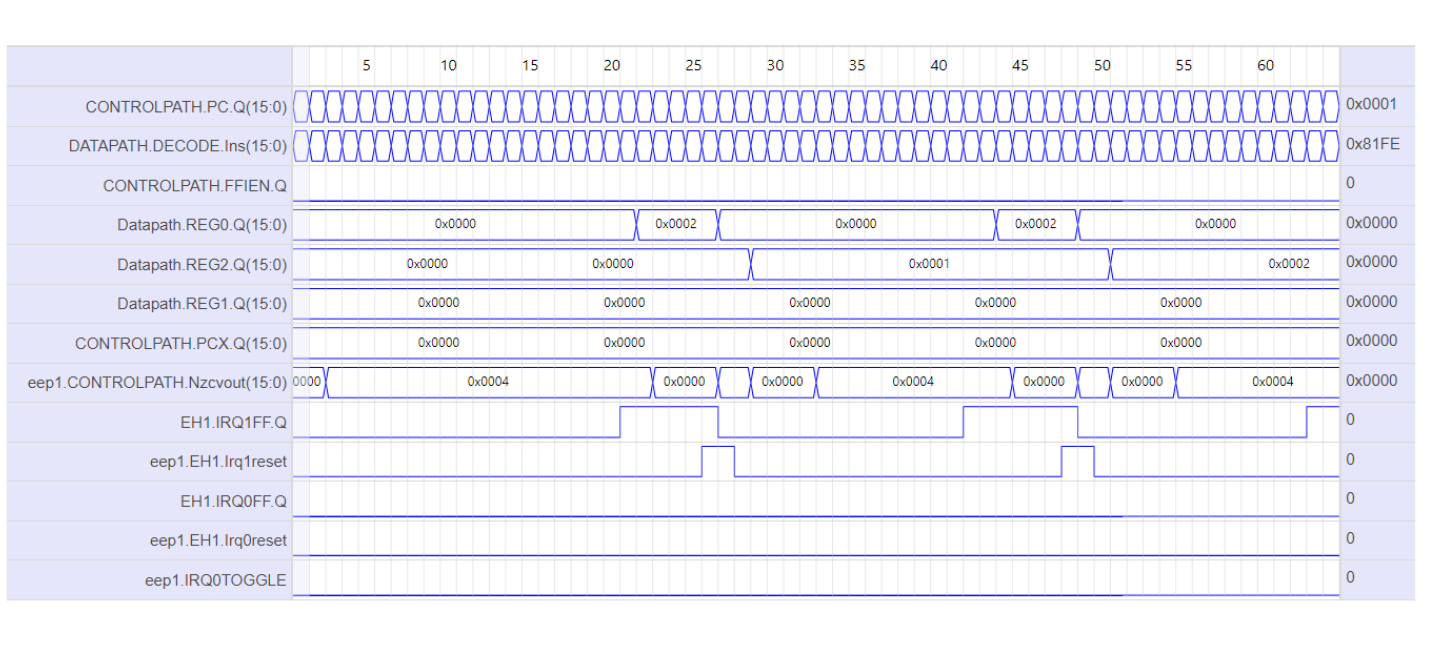


```
int-test2.txt
          // test interrupt 0 & 1
          // interrupts are not nested.
          // isr has conditional jump to determine which interrupt
          // is taken.
          // Base program
          SETI // set FFIEN.Q (enable interrupts)
     LOOP ADD R0, #1 // increment R0
          JNE LOOP

          ORG 0xff00 // code from here goes into address 0xff00
     ISR  STR R0, [0xFF80] // save base program R0
          MOV R0, Flags // save base program flags
          STR R0, [0xFF81]
          LDR R0, [0xFFFE]
          AND R0, #2 // is IRQ1 active?
          JNE ISR1 // if so jump to ISR1
          // otherwise, ISR0 must be active.
     ISR0 MOV R0, #1
          STR R0, [0xFFFF] // Interrupt 0 reset high
          MOV R0, #0
          STR R1, [0xFFFF] // Interrupt 0 reset low
          // Payload for interrupt 0
          // increment mem[0xFF82] by 1
          ADD R3, #1
          // small loop to add some time to INT0
          MOV R0, #3
          SUB R0, #1
          JNE -1
          // end of payload
     INTEND0
          LDR R0, [0xFF81]
          MOV Flags, R0 // restore base program flags
          LDR R0, [0xFF80] // restore R0
          RETINT
     ISR1
          MOV R0, #2
          STR R0, [0xFFFF] // Interrupt 1 reset high
          MOV R0, #0
          STR R0, [0xFFFF] // Interrupt 1 reset low
          // Payload for interrupt 1
          ADD R4, #1
          // end of payload

     INTEND1
          LDR R0, [0xFF81]
          MOV Flags, R0 // restore base program flags
          LDR R0, [0xFF80] // restore base program R0
          RETINT // return to base program and set FFIEN
```

When simulating in Issie, we get:


5 10 15 20 25 30 35 40 45 50 55 60
CONTROLPATH.PC.Q(15:0)
DATAPATH.DECODE.Ins(15:0)
CONTROLPATH.FFIEN.Q
Datapath.REG0.Q(15:0)
Datapath.REG2.Q(15:0)
Datapath.REG1.Q(15:0)
CONTROLPATH.PCX.Q(15:0)
eep1.CONTROLPATH.Nzcvout(15:0)
EH1.IRQ1FF.Q
eep1.EH1.Irq1reset
EH1.IRQ0FF.Q
eep1.EH1.Irq0reset
eep1.IRQ0TOGGLE
0x0001
0x81FE